

Source: README.md

Legal Document Summarizer & Clause Analyzer

A local, production-style legal AI project that converts complex legal text into a structured analysis format.

This repository is tutorial-first and artifact-grounded: every reported result in this README comes from files currently present in `artifacts/` and `data/processed/`.

Why This Exists

Legal text is hard for generic models because of long context dependencies, nested exceptions, cross-references, specialized terminology, and high hallucination risk. The project demonstrates a practical path from baseline prompting to domain-adapted fine-tuning for legal analysis workflows.

What the System Produces

For each input legal text, the system returns a structured schema:

- ``executive_summary``
- ``executive_bullets``
- ``clause_explanations``
- ``risk_level`` and ``risk_rationale``
- ``liabilities``
- ``obligations``
- ``rights``
- ``restrictions``
- ``red_flags``
- ``recommended_review_areas``
- ``evidence_spans``

Schema type: LegalAnalysis in `src/legal_clause_analyzer/schemas.py`.

Real Pipeline (Code-Accurate)

End-to-end orchestration: `scripts/run_all.py`

1. `scripts/prepare_data.py`
2. `scripts/run_baselines.py`
3. `scripts/train_model.py`
4. `scripts/evaluate.py`
5. `scripts/run_inference.py`

6. scripts/generate_notebooks.py
7. scripts/execute_notebooks.py
8. scripts/validate_strict_run.py (strict mode)

Strict validation checks include:

- prediction files exist for all systems
- no fallback prediction rows
- non-empty judge metrics
- CUDA training confirmation
- inference strategy is `fine\_tuned\_adapter`
- executed notebook outputs exist

Validation script: scripts/validate_strict_run.py

Model Roles and Rationale

Configured in configs/default.yaml.

- `qwen3-embedding:4b`: embedding/retrieval utility model role
- `qwen3.5:4b`: baseline generation + judge model
- `granite4.1:3b`: baseline generation + judge + fine-tuning base
- `glm-ocr`: OCR extension role

Fine-tuning base model (HF): ibm-granite/granite-4.1-3b

Alternative fallback base model (HF): Qwen/Qwen3.5-4B

Dataset and Ground-Truth Constraints

Source: coastalcph/lex_glue

Used subsets:

- `ledgar`
- `unfair\_tos`
- `scotus`
- `eurlex`
- `case\_hold`

Artifact-backed dataset report: data/processed/dataset_report.json

Current processed row counts:

- train: `1540`
- validation: `97`
- test: `337`

Policy for targets:

- real LexGLUE data only
- deterministic mapping from labels/text cues to structured legal outputs
- no synthetic replacement dataset

TRL, PEFT, Unsloth: What Was Actually Used

Implementation: `src/legal_clause_analyzer/finetune/train.py`

- TRL used (`SFTTrainer`, `SFTConfig`)
- PEFT used (`prepare\_model\_for\_kbit\_training`, `LoraConfig`, `get\_peft\_model`)
- Unsloth attempted first but not installed in this environment

Runtime proof: `artifacts/models/finetuned_adapter/training_backend_report.json`

- `backend\_order`: `['unsloth', 'trl']`
- `selected\_backend`: `trl\_peft`
- `selected\_model`: `ibm-granite/granite-4.1-3b`

Current Measured Results (Artifact-Backed, 2026-06-22)

Sources:

- `artifacts/metrics/system_metrics.json``
- `artifacts/metrics/scoreboard.csv``
- `artifacts/metrics/judge_report.json``
- `artifacts/metrics/latency_report.json``
- `artifacts/metrics/hallucination_report.json``

Primary baseline comparator: `few_shot` (same top metrics as other non-fine-tuned baselines in this run).

Metric	Baseline (few_shot)	Fine-Tuned Adapter	Delta
--- ---	--- ---	--- ---	
ROUGE-L	0.0023148148	0.0253431362	+0.0230283214
BERTScore F1	0.8335364461	0.8351356983	+0.0015992522
Risk F1	0.1085271318	0.1502192982	+0.0416921665
Obligation F1	0.2500000000	0.2500000000	0.0000000000

| Liability F1 | 0.0000000000 | 0.0000000000 | 0.0000000000 |
| Red-Flag F1 | 1.0000000000 | 1.0000000000 | 0.0000000000 |

LLM-as-a-Judge (Mean Overall)

- Granite judge overall:
- baseline (`few\_shot`): `2.722222222`
- fine-tuned: `2.777777778`
- delta: `+0.055555556`
- Qwen judge overall:
- baseline (`few\_shot`): `3.0`
- fine-tuned: `3.0`
- delta: `0.0`

Latency (Mean Seconds)

- `granite\_zero\_shot`: `1.5934249199`
- `few\_shot`: `2.3021680878`
- `qwen\_zero\_shot`: `2.3150713524`
- `prompt\_only`: `2.3928487653`
- `fine\_tuned\_adapter`: `7.9773901738`

Hallucination Diagnostics

For all systems in this run (including fine-tuned):

- `unsupported\_claim\_rate = 0.0`
- `missing\_obligation\_rate = 1.0`
- `missing\_liability\_rate = 1.0`

Interpretation: unsupported claims are controlled in this setup, but obligation/liability recall remains a key failure area.

Important Observed Behavior in Inference Examples

Source: artifacts/inference/inference_examples.json

- inference strategy: `fine\_tuned\_adapter` for all examples
- analysis payloads show parser fallback style outputs ("No summary generated.", empty arrays) in the shipped examples

Implication: strict path and adapter loading are working, but generation quality on these examples still needs improvement.

Training Runtime and Resource Snapshot

Source: artifacts/metrics/training_runtime_report.json

- `cuda\_available`: `true`
- elapsed seconds: `146.639812707901`
- max CUDA memory allocated: `5549631488` bytes
- max CUDA memory reserved: `5775556608` bytes

Training metrics source: artifacts/models/finetuned_adapter/training_metrics.json

Project Layout

```
Legal-Document-Summarizer-Clause-Analyzer/  
■■■ configs/  
■■■ data/  
■■■ artifacts/  
■■■ docs/  
■■■ notebooks/  
■■■ scripts/  
■■■ src/legal_clause_analyzer/  
■■■ tests/
```

Run Locally

Environment

```
cd /home/ahmad/AI/Legal-Document-Summarizer-Clause-Analyzer  
uv venv --python 3.12.10  
source .venv/bin/activate  
uv sync
```

Models

```
ollama pull qwen3-embedding:4b  
ollama pull qwen3.5:4b  
ollama pull granite4.1:3b  
ollama pull glm-ocr
```

Full strict run

```
source .venv/bin/activate  
python scripts/run_all.py --config configs/default.yaml --strict-live
```

Tutorial and Documentation Index

- Documentation index: `docs/README.md`
- Full handbook: `docs/HANDBOOK.md`
- Zero-to-hero chapter tutorials: `docs/tutorials/`
- PDF documentation: `docs/documentation.pdf`

Reuse Targets

This component is designed to be reused in:

- legal research assistants
- contract review systems
- compliance monitoring systems
- legal risk scoring pipelines
- regulatory intelligence platforms

Limitations and Next Steps

Current gaps shown by real outputs:

- liability extraction remains weak ($F1 = 0.0$)
- obligation recall remains weak ($\text{missing_obligation_rate} = 1.0$)
- inference example quality indicates parser fallback behavior still dominates

Priority next improvements:

1. stronger obligation/liability supervision strategy
2. richer long-context legal chunking + aggregation
3. schema-constrained decoding improvements
4. calibrated abstention for uncertain cases

References Used for Implementation Decisions

- TRL docs: https://huggingface.co/docs/trl/sft_trainer
- PEFT quantization guide: https://huggingface.co/docs/peft/main/developer_guides/quantization
- Unsloth docs: <https://docs.unsloth.ai/>
- LexGLUE dataset card: https://huggingface.co/datasets/coastalcph/lex_glue

Legal Disclaimer

This project is for technical demonstration and workflow support. It is not legal advice.

Source: docs/HANDBOOK.md

Legal Document Summarizer & Clause Analyzer Handbook

This handbook explains the project end-to-end using the actual implemented code and real executed artifacts in this repository.

It is designed for:

- beginners learning legal AI engineering from scratch
- intermediate/advanced engineers who need implementation-level detail
- teams that want a reusable, local, auditable legal analysis component

Table of Contents

1. Problem and objectives
2. Repository architecture and workflow
3. Data pipeline and supervision design
4. Baselines and prompting systems
5. Fine-tuning system (TRL + PEFT + optional Unsloth)
6. Evaluation framework and judge system
7. Error analysis and observed failure modes
8. Inference pipeline and production usage
9. Strict run validation and reproducibility
10. Real results interpretation and practical takeaways
11. References and appendix

1) Problem and Objectives

Problem statement

Legal text understanding is difficult because obligations, liabilities, restrictions, and risk are often encoded in long, nested, cross-referenced language. Generic summarization is insufficient when the output must be legally structured and operationally useful.

Project objective

Build a local system that outputs structured legal analysis for legal text:

- plain-English summary
- executive bullets
- clause-level explanations
- risk classification

- liability extraction
- obligation/right/restriction extraction
- red-flag detection
- recommended review areas

Output contract in code

Defined in `src/legal_clause_analyzer/schemas.py` as `LegalAnalysis`.

2) Repository Architecture and Workflow

What is this technique?

A staged legal AI pipeline with explicit separation of data preparation, baseline benchmarking, fine-tuning, evaluation, and inference.

Why is it used?

Staging isolates failure points and gives auditable before-vs-after evidence (baseline vs fine-tuned).

How it appears in the code

- orchestrator: ``scripts/run_all.py``
- strict validator: ``scripts/validate_strict_run.py``
- architecture note: ``docs/ARCHITECTURE.md``

Practical workflow (actual run order)

1. `scripts/prepare_data.py`
2. `scripts/run_baselines.py`
3. `scripts/train_model.py`
4. `scripts/evaluate.py`
5. `scripts/run_inference.py`
6. `scripts/generate_notebooks.py`
7. `scripts/execute_notebooks.py`
8. `scripts/validate_strict_run.py`

3) Data Pipeline and Supervision Design

What is this technique?

Deterministic policy-mapped supervision from real legal datasets.

Why is it used?

LexGLUE provides real legal labels, but not a single ready-made schema for executive summary + risk + liabilities + obligations. Deterministic mapping creates unified training targets without synthetic dataset replacement.

How it appears in the code

- data ingestion and split preservation: ``src/legal_clause_analyzer/data/lexglue_loader.py``
- policy mapping rules: ``src/legal_clause_analyzer/data/policy_mapping.py``
- row transformation to supervised schema: ``src/legal_clause_analyzer/data/instruction_builder.py``
- split write + dataset report: ``scripts/prepare_data.py``

Practical details from real artifacts

Source: `data/processed/dataset_report.json`

- train rows: ``1540``
- validation rows: ``97``
- test rows: ``337``

Included LexGLUE subsets:

- ``ledgar``
- ``unfair_tos``
- ``scotus``
- ``eurlex``
- ``case_hold``

Current dataset limitations (artifact-backed):

- no native executive-summary labels across all tasks
- no direct gold risk/liability taxonomy for the unified output schema
- cross-jurisdiction semantics remain challenging for compact local models

4) Baselines and Prompting Systems

What is this technique?

Baseline benchmarking with prompt variants before any fine-tuning.

Why is it used?

Without baselines, improvement claims are not meaningful.

How it appears in the code

- prompt templates and JSON parser fallback: ``src/legal_clause_analyzer/baselines/generation.py``
- model clients (``ollama``, ``transformers``): ``src/legal_clause_analyzer/baselines/model_clients.py``
- baseline runner: ``scripts/run_baselines.py``

Baselines implemented

- ``prompt_only``
- ``few_shot``
- ``granite_zero_shot``
- ``qwen_zero_shot``

Practical note from predictions

Current baseline prediction files are in `artifacts/runs/balanced_local_v1/predictions_*.jsonl`.

In this run, all baseline systems produced similar top-line metrics (see section 10).

5) Fine-Tuning System (TRL + PEFT + optional Unsloth)

What is this technique?

Parameter-efficient supervised fine-tuning (LoRA/QLoRA) on legal instruction-response rows.

Why is it used?

Full-model fine-tuning is usually impractical on local 8GB-class GPUs. QLoRA enables adapter tuning with lower VRAM footprint.

How it appears in the code

- training implementation: ``src/legal_clause_analyzer/finetune/train.py``
- SFT text format: ``src/legal_clause_analyzer/finetune/dataset_format.py``
- training entrypoint and GPU/runtime report: ``scripts/train_model.py``

TRL

- definition: training framework for LLM fine-tuning with ``SFTTrainer``
- where used: ``SFTConfig``, ``SFTTrainer`` in ``train.py``
- what changed: structured trainer/eval/checkpoint flow instead of ad-hoc loops

PEFT

- definition: adapter-based parameter-efficient tuning

- where used: ``prepare_model_for_kbit_training`, `LoraConfig`, `get_peft_model``
- what changed: feasible 4-bit adapter training on local hardware

Unsloth

- definition: optional acceleration backend for LoRA/QLoRA workflows
- where used: optional ``_load_unsloth_bundle`` path in ``train.py``
- what changed in this environment: attempted first, not available, clean fallback to TRL+PEFT

Runtime proof: `artifacts/models/finetuned_adapter/training_backend_report.json`

- ``backend_order`: `['unsloth', 'trl']``
- ``selected_backend`: `trl_peft``
- ``selected_model`: `ibm-granite/granite-4.1-3b``

Practical resource/runtime snapshot

Source: `artifacts/metrics/training_runtime_report.json`

- ``cuda_available`: `true``
- elapsed: ``146.639812707901`` seconds
- max CUDA allocated: ``5549631488`` bytes
- max CUDA reserved: ``5775556608`` bytes

6) Evaluation Framework and Judge System

What is this technique?

Multi-axis evaluation combining automatic metrics, extraction/classification scores, hallucination diagnostics, latency, and LLM-as-a-judge.

Why is it used?

Legal systems need both text quality and structured legal utility. A single metric is insufficient.

How it appears in the code

- metric computation: ``src/legal_clause_analyzer/evaluation/metrics.py``
- hallucination diagnostics: ``src/legal_clause_analyzer/evaluation/hallucination.py``
- judge logic: ``src/legal_clause_analyzer/evaluation/judge.py``
- plotting: ``src/legal_clause_analyzer/evaluation/plots.py``
- evaluator orchestration: ``scripts/evaluate.py``

Metrics implemented

Summarization:

- ROUGE-1
- ROUGE-2
- ROUGE-L
- BLEU
- METEOR
- BERTScore

Classification / extraction:

- risk: accuracy, precision, recall, F1
- obligation extraction: precision, recall, F1
- liability extraction: precision, recall, F1
- red-flag extraction: precision, recall, F1

Judge dimensions:

- legal accuracy
- summary quality
- risk identification quality
- clause extraction quality
- hallucination risk
- executive usefulness
- overall

7) Error Analysis and Observed Failure Modes

What is this technique?

Artifact-driven post-run failure analysis.

Why is it used?

To move from "did metrics go up" to "what is still broken and why".

How it appears in code and artifacts

- hallucination diagnostics code: ``evaluation/hallucination.py``
- reports: ``artifacts/metrics/hallucination_report.json``
- confusion and score plots: ``artifacts/figures/risk_confusion_matrix.png``,
``artifacts/figures/system_scoreboard.png``

Real observed failure pattern in this run

- ``unsupported_claim_rate`` is low (``0.0``), but
- ``missing_obligation_rate`` and ``missing_liability_rate`` remain ``1.0`` across systems

Interpretation: strict unsupported claim control is not translating into recall for obligation/liability extraction.

8) Inference Pipeline and Production Usage

What is this technique?

Adapter-first inference with controlled fallback behavior.

Why is it used?

Provides a reliable production path:

- use fine-tuned adapter when available
- keep strict mode for validation workflows
- permit fallback behavior in non-strict runtime scenarios

How it appears in the code

- main inference logic: ``src/legal_clause_analyzer/inference/pipeline.py``
- inference script: ``scripts/run_inference.py``

Practical behavior from current artifacts

Source: `artifacts/inference/inference_examples.json`

- strategy used: ``fine_tuned_adapter`` for all examples
- model used: ``ibm-granite/granite-4.1-3b``
- output content shows parser-fallback style values in examples (``"No summary generated."`` + mostly empty structured fields)

This indicates runtime path correctness but quality bottlenecks in generation/parsing for these examples.

9) Strict Run Validation and Reproducibility

What is this technique?

Post-run integrity checks to ensure outputs are real, complete, and fallback-free in strict mode.

Why is it used?

Legal AI claims require auditable run integrity.

How it appears in the code

scripts/validate_strict_run.py verifies:

- required predictions exist
- no fallback summaries in predictions
- judge report entries exist and are non-empty
- CUDA availability in training runtime report
- inference strategies are fine-tuned adapter only
- executed notebooks exist (when required)

10) Real Results Interpretation and Practical Takeaways

Key results (artifact-backed)

Sources:

- ``artifacts/metrics/system_metrics.json``
- ``artifacts/metrics/scoreboard.csv``
- ``artifacts/metrics/judge_report.json``
- ``artifacts/metrics/latency_report.json``

Fine-tuned adapter vs baseline (few_shot):

- ROUGE-L: ``0.0253431362`` vs ``0.0023148148`` (``+0.0230283214``)
- BERTScore F1: ``0.8351356983`` vs ``0.8335364461`` (``+0.0015992522``)
- Risk F1: ``0.1502192982`` vs ``0.1085271318`` (``+0.0416921665``)
- Obligation F1: tie at ``0.25``
- Liability F1: tie at ``0.0``
- Red-Flag F1: tie at ``1.0``

Judge mean overall:

- granite judge: ``2.7777777778`` (fine-tuned) vs ``2.7222222222`` (baseline)
- qwen judge: ``3.0`` vs ``3.0`` (tie)

Latency mean (seconds):

- fine-tuned adapter: ``7.9773901738``
- baseline systems: ``1.5934`` to ``2.3928``

Practical takeaway

Fine-tuning improves summary and risk signals in this run, but liability and obligation extraction recall remain the main technical gap. The system is functionally complete and reproducible, but not yet strong enough to skip downstream legal review.

11) References and Appendix

Official references used in implementation decisions

- TRL SFT docs: https://huggingface.co/docs/trl/sft_trainer
- PEFT quantization guide: https://huggingface.co/docs/peft/main/developer_guides/quantization
- Unsloth docs: <https://docs.unsloth.ai/>
- LexGLUE dataset card: https://huggingface.co/datasets/coastalcph/lex_glue

Related project files

- config: ``configs/default.yaml``
- architecture summary: ``docs/ARCHITECTURE.md``
- notebook sequence: ``notebooks/01_*.py`` through ``notebooks/07_*.py``

Legal disclaimer

This project is a technical implementation and educational resource. It is not legal advice.

Tutorial 01: Problem Setup and Legal Challenges

Learning Goals

By the end of this chapter, you should understand:

- why legal summarization and clause analysis are hard
- why domain-specific adaptation matters
- where this intent is encoded in the current repository

What Is This Technique?

Definition

Problem framing for legal AI means converting a broad goal ("summarize legal text") into explicit, testable capabilities with a stable output schema.

Why It Is Used

Without strong framing, legal models drift into generic text generation and fail at operational needs (risk labels, obligations, liabilities, red flags).

How It Appears in the Code

- output schema and contract: ``src/legal_clause_analyzer/schemas.py``
- high-level architecture: ``docs/ARCHITECTURE.md``
- end-to-end pipeline steps: ``scripts/run_all.py``
- strict completion checks: ``scripts/validate_strict_run.py``

Practical Explanation in This Project

This project defines a single schema (LegalAnalysis) and makes every stage (data, baselines, training, evaluation, inference) produce or consume that schema. That is the core reason metrics and artifacts are comparable across systems.

Why Legal Summarization Is Difficult

- long-range dependencies across clauses
- nested exceptions and carve-outs
- jurisdiction-specific legal language
- ambiguous terms requiring context
- high cost of hallucinated obligations/liabilities

These constraints are reflected in the pipeline design:

- deterministic supervision mapping (for consistency)
- strict schema parsing (for structure)
- multi-axis evaluation (for realism)
- hallucination diagnostics (for safety)

General LLMs vs Domain-Tuned Legal Models

General models (prompt-only)

- easy to start
- often inconsistent in legal structure extraction
- weak repeatability for legal risk workflows

Domain-tuned models (this project)

- higher structure adherence
- improved risk scoring metrics in this run
- still limited on liability recall in current data/training setup

Project Objective as Implemented

Inputs:

- legal clauses, legal opinions, regulations, legal text passages

Outputs:

- structured legal analysis fields in `LegalAnalysis`

Validation approach:

- baseline-first benchmarking
- fine-tuned comparison
- judge scoring and hallucination diagnostics
- strict-run validation

Real-Run Context You Should Keep in Mind

From artifacts/metrics/system_metrics.json and related reports:

- summary and risk metrics improved after fine-tuning

- obligation/liability recall remains a major gap

This chapter establishes why both can be true at the same time.

Key Takeaway

The project is not a generic summarizer demo. It is a schema-constrained legal analysis pipeline with measurable behavior and explicit known limitations.

Source: docs/tutorials/02_dataset_and_supervision_design.md

Tutorial 02: Dataset and Supervision Design

Learning Goals

- understand exactly how real LexGLUE data is ingested
- understand deterministic label-to-schema mapping
- connect data design decisions to observed model behavior

What Is This Technique?

Definition

Deterministic supervision design maps raw dataset labels and text cues to a unified instruction-response schema for training and evaluation.

Why It Is Used

LexGLUE tasks are diverse and label spaces differ. A unified schema is required to train one model for executive summary + risk + extraction tasks.

How It Appears in the Code

- dataset loader and split handling: ``src/legal_clause_analyzer/data/lexglue_loader.py``
- policy-based mapping functions: ``src/legal_clause_analyzer/data/policy_mapping.py``
- supervision row builder: ``src/legal_clause_analyzer/data/instruction_builder.py``
- persisted dataset report + distributions: ``scripts/prepare_data.py``

Practical Explanation in This Project

`prepare_data.py` writes `train_supervised.jsonl`, `validation_supervised.jsonl`, and `test_supervised.jsonl` and a `dataset_report.json` that captures row counts, legal category hierarchy, and limitations.

Dataset Ground Truth

Source dataset: `coastalcph/lex_glue`

Used subsets:

- ``ledgar``
- ``unfair_tos``
- ``scotus``
- ``eurlex``

- ``case_hold``

Current processed row counts (data/processed/dataset_report.json):

- train: ``1540``
- validation: ``97``
- test: ``337``

Labeling Schema and Hierarchy

From artifact report:

- single-label datasets: ``ledger``, ``scotus``, ``case_hold``
- multi-label datasets: ``unfair_tos``, ``eurlex``

Regulatory hierarchy used in docs/report:

- contract law: ``ledger``, ``unfair_tos``
- US case law: ``scotus``, ``case_hold``
- EU regulatory law: ``eurlex``

Long-Text Handling in Current Implementation

What Is This Technique?

Conservative truncation and normalization for local runtime stability.

Why It Is Used

Prevents memory and latency blow-ups on local hardware.

How It Appears in the Code

- text normalization and truncation: ``clean_text`` in ``lexglue_loader.py``
- max input char control: ``configs/default.yaml`` (``sampling.max_input_chars``)

Practical Note

This is a pragmatic local strategy, not a perfect long-document strategy. For stronger long-context legal performance, add chunking/sliding windows + aggregation in a future version.

Dataset Limitations (Artifact-Backed)

From dataset_report.json:

- no native gold executive-summary labels for all tasks
- no direct gold risk/liability taxonomy for unified schema
- cross-jurisdiction semantics can be underrepresented in compact model training

Key Takeaway

The supervision layer is auditable and deterministic, which improves reproducibility, but it is also the main bottleneck behind extraction recall limits.

Source: docs/tutorials/03_baselines_and_prompting_systems.md

Tutorial 03: Baselines and Prompting Systems

Learning Goals

- understand baseline systems and prompt styles
- understand why baseline-first evaluation is mandatory
- map prompt behavior to current metrics

What Is This Technique?

Definition

Baseline benchmarking is the controlled evaluation of non-fine-tuned systems before adaptation.

Why It Is Used

It defines the reference point for all improvement claims.

How It Appears in the Code

- baseline specs and prompt construction: ``src/legal_clause_analyzer/baselines/generation.py``
- generation backends and parsing: ``src/legal_clause_analyzer/baselines/model_clients.py``
- baseline execution script: ``scripts/run_baselines.py``

Practical Explanation in This Project

The project runs four baseline systems on held-out test rows and writes per-system predictions_*.jsonl files with latency measurements.

Baseline Systems Implemented

- ``prompt_only``
- ``few_shot``
- ``granite_zero_shot``
- ``qwen_zero_shot``

Files:

- ``artifacts/runs/balanced_local_v1/predictions_prompt_only.jsonl``
- ``artifacts/runs/balanced_local_v1/predictions_few_shot.jsonl``
- ``artifacts/runs/balanced_local_v1/predictions_granite_zero_shot.jsonl``

- ``artifacts/runs/balanced_local_v1/predictions_qwen_zero_shot.jsonl``

Prompt and Parsing Strategy

What Is This Technique?

Schema-constrained prompting + best-effort JSON parsing.

Why It Is Used

Keeps output close to the legal schema and reduces free-form drift.

How It Appears in the Code

- system prompt requires JSON keys: ``SYSTEM_PROMPT`` in ``generation.py``
- parser fallback logic: ``try_parse_json`` in ``model_clients.py``
- safe conversion to ``LegalAnalysis``: ``safe_analysis_from_text`` in ``generation.py``

Practical Observation

Current predictions include many parser-fallback style outputs ("No summary generated."), which explains weak extraction recall despite stable run completion.

Real Baseline Performance Snapshot

From `artifacts/metrics/system_metrics.json`:

- baseline ROUGE-L around ``0.0023148148``
- baseline risk F1 around ``0.1085271318``

These are the values the fine-tuned model is compared against.

Key Takeaway

Baselines reveal that local prompt-only/legal-prompt systems can run reliably but produce limited legal extraction quality without adaptation.

Source: docs/tutorials/04_finetuning_with_trl_peft_unsloth.md

Tutorial 04: Fine-Tuning with TRL, PEFT, and Unsloth

Learning Goals

- understand LoRA/QLoRA in this codebase
- understand exactly why TRL and PEFT are required
- understand where Unsloth is optional and what happened in this run

What Is This Technique?

Definition

QLoRA fine-tuning trains small adapter parameters on top of a quantized base model, rather than full-model parameter updates.

Why It Is Used

It makes local fine-tuning feasible on limited VRAM GPUs.

How It Appears in the Code

- training pipeline core: ``src/legal_clause_analyzer/finetune/train.py``
- trainer entrypoint and GPU guard logic: ``scripts/train_model.py``
- training format: ``src/legal_clause_analyzer/finetune/dataset_format.py``

Practical Explanation in This Project

The training stack tries Unsloth first (if available), then falls back to TRL+PEFT. In this environment, Unsloth was unavailable, and training completed with trl_peft.

TRL Coverage

Definition

TRL provides SFTTrainer and SFTConfig for supervised instruction tuning.

Why used here

Needed for stable trainer lifecycle: logging, eval, save, and metric traces.

Where used

SFTTrainer/SFTConfig usage in train.py.

Post-run impact

Produced structured training artifacts:

- `training\_metrics.json`
- `training\_curve.csv`
- adapter checkpoint and metadata files

PEFT Coverage

Definition

PEFT (Parameter-Efficient Fine-Tuning) attaches LoRA adapters.

Why used here

Avoids full-parameter fine-tuning cost on local hardware.

Where used

- `prepare\_model\_for\_kbit\_training`
- `LoraConfig`
- `get\_peft\_model`

all in train.py.

Post-run impact

Fine-tuned adapter artifacts created under artifacts/models/finetuned_adapter/ with quantized_4bit=true in training metrics.

Unsloth Coverage

Definition

Optional accelerated backend for LoRA/QLoRA flows.

Why considered

Potential speed and memory improvements.

Where used

Optional path `_load_unsloth_bundle` in train.py.

Post-run impact

Source: artifacts/models/finetuned_adapter/training_backend_report.json

- Unsloth attempts recorded
- both attempts failed with "Unsloth is not installed in this environment"
- selected backend: `trl\_peft`

Real Training Runtime Snapshot

Source: artifacts/metrics/training_runtime_report.json

- CUDA available: `true`
- elapsed: `146.639812707901s`
- max allocated: `5549631488` bytes
- max reserved: `5775556608` bytes

Key Takeaway

TRL+PEFT is the required backbone in this environment; Unsloth is an optimization path, not a hard dependency.

Source: docs/tutorials/05_evaluation_and_llm_judging.md

Tutorial 05: Evaluation and LLM-as-a-Judge

Learning Goals

- understand every metric family used
- understand how judge scoring is implemented
- connect metrics to real artifact outputs

What Is This Technique?

Definition

A multi-dimensional evaluation stack combining automatic metrics, extraction/classification metrics, hallucination diagnostics, latency, and LLM-as-a-judge scoring.

Why It Is Used

Legal AI quality is multi-objective. Good summary text alone does not imply good risk or liability extraction.

How It Appears in the Code

- automatic metrics: ``src/legal_clause_analyzer/evaluation/metrics.py``
- judge prompt and parsing: ``src/legal_clause_analyzer/evaluation/judge.py``
- hallucination diagnostics: ``src/legal_clause_analyzer/evaluation/hallucination.py``
- evaluation orchestration: ``scripts/evaluate.py``

Practical Explanation in This Project

evaluate.py reads all prediction files, computes metrics, runs two judge models, writes report JSON files, and creates visualization figures.

Metric Families

Summarization:

- ROUGE-1/2/L
- BLEU
- METEOR
- BERTScore

Classification/extraction:

- risk accuracy/precision/recall/F1
- obligation precision/recall/F1
- liability precision/recall/F1
- red-flag precision/recall/F1

Judge dimensions:

- legal accuracy
- summary quality
- risk identification quality
- clause extraction quality
- hallucination risk
- executive usefulness
- overall

Real Results (Fine-Tuned vs Baseline)

From artifacts/metrics/system_metrics.json and scoreboard.csv:

- ROUGE-L: `0.0253431362` vs `0.0023148148`
- BERTScore F1: `0.8351356983` vs `0.8335364461`
- Risk F1: `0.1502192982` vs `0.1085271318`

From artifacts/metrics/judge_report.json:

- granite judge overall: `2.7777777778` vs `2.7222222222`
- qwen judge overall: tie at `3.0`

Latency View

From artifacts/metrics/latency_report.json:

- fine-tuned mean latency: `7.9773901738s`
- baseline means: `1.5934s` to `2.3928s`

This highlights the speed-quality tradeoff for adapter inference.

Key Takeaway

Evaluation must be interpreted as a bundle. In this run, summary and risk improved, but extraction recall remains the dominant limitation.

Source: docs/tutorials/06_error_analysis_and_visualization.md

Tutorial 06: Error Analysis and Visualization

Learning Goals

- identify concrete failure patterns from current artifacts
- interpret visual diagnostics correctly
- map failures back to implementation constraints

What Is This Technique?

Definition

Structured error analysis is the process of converting raw predictions and metrics into actionable model improvement hypotheses.

Why It Is Used

Without error analysis, you only know whether metrics changed, not what to fix next.

How It Appears in the Code

- hallucination metrics logic: ``src/legal_clause_analyzer/evaluation/hallucination.py``
- plotting functions: ``src/legal_clause_analyzer/evaluation/plots.py``
- metric/judge aggregation for analysis: ``scripts/evaluate.py``

Practical Explanation in This Project

The evaluation stage writes machine-readable reports and figures to `artifacts/metrics/` and `artifacts/figures/`, enabling repeatable post-run diagnosis.

Real Failure Signals in This Run

From `artifacts/metrics/hallucination_report.json`:

- unsupported claim rate: ``0.0``
- missing obligation rate: ``1.0``
- missing liability rate: ``1.0``

Interpretation:

- the system avoids adding unsupported extraction items
- but frequently fails to recover required obligation/liability items

Visualization Assets

- `artifacts/figures/training\_curve.png`
- `artifacts/figures/metric\_bars.png`
- `artifacts/figures/system\_scoreboard.png`
- `artifacts/figures/risk\_confusion\_matrix.png`
- `artifacts/figures/dataset\_distributions.png`

Root-Cause Hypotheses (Implementation-Consistent)

1. supervision mapping provides conservative signals but limited fine-grained liability labels
2. structured parser fallback (safe_analysis_from_text) can collapse outputs into empty fields
3. compact model size + short sequence constraints limit nuanced extraction retention

Key Takeaway

The next gains are likely to come from better supervision richness and decoding/parsing robustness, not only from repeating the current training recipe.

Source: docs/tutorials/07_inference_workflow_and_examples.md

Tutorial 07: Inference Workflow and Example Outputs

Learning Goals

- understand adapter-first inference behavior
- understand strict vs non-strict inference behavior
- interpret current example outputs correctly

What Is This Technique?

Definition

Adapter-first inference tries the fine-tuned adapter path first, then uses fallback paths depending on strictness and runtime conditions.

Why It Is Used

Balances production reliability (fallbacks) with strict validation integrity (fail-fast when required).

How It Appears in the Code

- inference orchestration: ``src/legal_clause_analyzer/inference/pipeline.py``
- runtime script for examples: ``scripts/run_inference.py``
- strict behavior tests: ``tests/test_strict_inference.py``

Practical Explanation in This Project

The script runs three legal examples and writes `artifacts/inference/inference_examples.json`, including model used, strategy, latency, and full structured output.

Current Real Example Outcomes

Source: `artifacts/inference/inference_examples.json`

- strategy for all three examples: ``fine_tuned_adapter``
- model for all three examples: ``ibm-granite/granite-4.1-3b``
- outputs show parser fallback style values:
- ``executive_summary``: ``"No summary generated."``
- empty extraction arrays
- default risk rationale fallback text

Why This Matters

Pipeline routing is functioning correctly (adapter path selected), but output quality on these examples indicates unresolved generation/parsing robustness issues.

Production Integration Notes

The reusable function is `analyze_legal_text` in `inference/pipeline.py`.

`InferenceResult` returns:

- ``analysis`` (structured ``LegalAnalysis``)
- ``model_used``
- ``strategy``

This is directly usable as a backend component in larger legal AI systems.

Key Takeaway

System-level integration is complete and strict-path compatible, but model output quality still requires targeted improvement work.

Tutorial 08: Strict Run Validation and Artifact Audit

Learning Goals

- understand strict run guarantees in this project
- know exactly what is verified post-run
- use artifact checks for reproducibility and auditability

What Is This Technique?

Definition

Strict-run validation is a post-execution integrity gate that verifies artifact completeness and disallows hidden fallback behavior.

Why It Is Used

In legal workflows, reproducibility and evidence integrity are as important as raw performance.

How It Appears in the Code

- validator script: ``scripts/validate_strict_run.py``
- strict flags in orchestrators:
- ``scripts/run_all.py``
- ``scripts/run_baselines.py``
- ``scripts/train_model.py``
- ``scripts/evaluate.py``
- ``scripts/run_inference.py``

Practical Explanation in This Project

The strict validator checks:

- all prediction files exist
- zero fallback rows in predictions
- judge report entries are non-empty
- CUDA availability recorded in training runtime
- inference strategy is ``fine_tuned_adapter``
- executed notebook outputs exist (when required)

Real Artifact Audit Checklist

Core reports:

- `artifacts/metrics/system\_metrics.json`
- `artifacts/metrics/judge\_report.json`
- `artifacts/metrics/latency\_report.json`
- `artifacts/metrics/hallucination\_report.json`
- `artifacts/metrics/training\_runtime\_report.json`

Predictions:

- `artifacts/runs/balanced\_local\_v1/predictions\_\*.jsonl`

Inference examples:

- `artifacts/inference/inference\_examples.json`

Notebook execution outputs:

- `notebooks/\*.executed.ipynb`

Interpreting a Passing Strict Run

A strict pass means:

- the workflow completed end-to-end
- expected outputs exist and are structurally valid
- no hidden deterministic fallback rows were accepted in strict prediction paths

It does not mean legal quality is solved. Quality still depends on the metric profile and error analysis outcomes.

Key Takeaway

Strict validation provides trust in run integrity; it is the foundation for credible comparison and iterative model improvement.